

TP7 – CNN

Ce TP sera réalisé sous tensorflow2 et keras. La documentation pourra être trouvée sous https://www.tensorflow.org/api_docs/python/tf/keras/

I. Chargement et mise en forme des données

Etudier et exécuter le programme donné. Vous utiliserez les 3 derniers chiffres de votre carte d'étudiant pour définir la variable `random_state`

Questions

Que réalise-t-il ? Justifiez chaque ligne.

Combien y a-t-il d'images dans la base de test ? Dans la base d'apprentissage ? Quelle est la taille des images ? Combien y a-t-il de classes ?

En quoi consiste le pré-traitement des données d'entrée ? Pourquoi le réalise-t-on ?

A quoi sert la fonction `tf.keras.utils.to_categorical` ? Quelle est la taille de `y_train` ? de `Y_train` ? Commenter.

II. Régression logistique

a. II.1. Définition du réseau

On souhaite classifier les données d'entrée en 10 classes et pour cela, on estime la sortie comme une somme pondérée des valeurs entrées. On estime les poids par apprentissage d'un réseau de neurones à une couche.

Questions

Pourquoi utilise-t-on la fonction d'activation softmax ?

Combien y a-t-il de paramètres à apprendre (Trouver les par le calcul puis vérifier) ?

A quoi sert la commande Flatten ?

b. II.2. Apprentissage

On utilise l'optimiseur SGD (Stochastic gradient descent optimizer) avec ses paramètres par défaut

Questions

Que représente `lr`, `batch` et `epochs` ?

Pourquoi utilise-t-on '`categorical_crossentropy`' comme fonction perte ?

Commenter ligne à ligne le programme. Quelle fonction perte est utilisée ?

Que réalise la fonction `affiche(history)` donnée en annexe ? Est-ce que l'apprentissage se passe bien ?

c. II.3. Evaluation du modèle

Questions

Que renvoie `model.predict` et pourquoi utilise-t-on `y_pred.argmax` ?

Refaites l'apprentissage en réglant au mieux les valeurs de `lr`, `batch` et `epochs`. Quel est le meilleur taux de reconnaissance obtenu ?

III. MLP

Reprenez toutes les questions précédentes en ajoutant une couche cachée avec 256 neurones et la fonction d'activation adéquate.

Questions

Combien y a-t-il de paramètres à estimer (trouver les par le calcul puis vérifier) ?

Changez les valeurs du pas d'apprentissage, du momentum, du `batch_size` pour optimiser les performances du réseau (les valeurs par défaut sont `lr=0.01`, `momentum=0.0`).

Que se passe-t-il si le pas d'apprentissage est trop grand ? Trop petit ? Comment agit le momentum ?

Ajoutez une couche de dropout. Comment agit-elle ?

Ajoutez une seconde couche cachée. Que se passe-t-il ?

Conclusion.

IV. CNN

Gardez les paramètres d'apprentissage optimaux et définissez un réseau en ajoutant avant les couches du MLP :

- une couche convolutionnelle composée de 32 filtres 3x3 suivie de la fonction d'activation RELU
- une couche convolutionnelle composée de 64 filtres 3x3 suivie de la fonction d'activation RELU

Questions

Combien y a-t-il de paramètres à estimer (trouver les par le calcul puis vérifier)? Comment est la convergence du réseau ? Est-elle plus rapide, plus lente ? Comment sont les résultats ?

Pour être plus robuste aux translations, ajoutez un max-pooling de taille 2 après la dernière couche convolutionnelle.

Pour améliorer la généralisation, ajoutez une ou plusieurs couches de dropout et reprenez les mêmes questions.

Essayez différentes architectures du réseau pour optimiser les résultats.

Sur ces images, il n'y a pas grand intérêt à utiliser un CNN plutôt qu'un MLP. Pourquoi ?
Quelle va être la force des CNN ?